

FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

Project Title: TeamRun - Fitness Application

Team Members:

- 1. Daniel Herrera (Team Leader)**
- 2. Carlos Martinez**
- 3. Anthony Jimenez**
- 4. Harut Kulakchian**
- 5. Osbaldo Bravo**

1. Introduction

Our app seeks to make fitness more fun and easy to do. The way our app does this is by making workouts increase your characters stats, and the higher stats you have, the better the player is suited for battle.

2. System Overview

The system is a web-based application developed to transform fitness tracking into an engaging game experience where users

- Complete real-world workouts to level up their character
- Battle enemies
- Track their fitness progress.

3. Functional Requirements

User Management

FR-1

The system shall allow users to register for a new account with email, password, and hero name.

FR-2

The system shall allow users to log in to their existing account using email and password.

FR-3

The system shall authenticate users using JSON Web Tokens (JWT) for secure session management.

FR-4

The system shall allow users to view their profile with character statistics and progress.

Fitness Tracking

FR-5

The system shall allow users to log water intake in cups with corresponding XP rewards.

FR-6

The system shall display today's water consumption and progress toward the daily goal of 8 cups.

FR-7

The system shall allow users to log workouts including exercise name, repetitions, weight, and number of sets.

FR-8

The system shall calculate XP based on workout intensity

FR-9

The system shall store workout history for each user with timestamps.

Gamification

FR-10

The system shall track user XP and automatically level up when reaching level thresholds.

FR-11

The system shall increase character stats (Strength, Stamina, Agility) when leveling up.

FR-12

The system shall display character statistics on the user dashboard.

FR-13

The system shall track user gold balance earned through battles.

Battle System

FR-14

The system shall allow users to start battles against random enemies from a predefined list.

FR-15

The system shall calculate battle damage based on workout history and character strength.

FR-16

The system shall apply permanent health damage to the user when attacked by enemies.

FR-17

The system shall allow users to flee from battles with a small health penalty.

FR-18

The system shall award XP and gold for defeating enemies.

FR-19

The system shall store battle history with outcomes (victory/defeat).

FR-20

The system shall allow users to purchase health potions using gold to restore health.

Quests

FR-21

The system shall assign active quests to users based on workout and hydration goals.

FR-22

The system shall track quest progress automatically based on user activities.

FR-23

The system shall allow users to claim quest rewards upon completion.

FR-24

The system shall display available quests with progress bars and reward information.

Dashboard & Reporting

FR-25

The system shall display recent user activities including workouts, battles, and water intake.

FR-26

The system shall show level progress with XP bar and next level requirements.



FR-27

The system shall display character stat progression with visual indicators.

FR-28

The system shall show battle history with win/loss records.

4. Feature Classification

Required Features (Core functionality necessary for the system to operate)

- User registration and login with JWT authentication

- Workout logging with XP calculation
- Water intake tracking
- Character stats display (Strength, Stamina, Agility)
- Level progression system
- Battle system with health management
- Gold rewards and spending
- MongoDB database for persistent data storage
- RESTful API endpoints for all operations

Desired Features (Additional features that improve usability and performance)

Active quests with progress tracking

- Battle history and workout history display
- Shop for purchasing health potions
- Recent activity feed on dashboard
- Enemy variety with different difficulties
- Random enemy selection for battles
- Responsive design using Tailwind CSS • Hero name personalization

Aspirational Features (Advanced features that enhance system value)

- Real-time battle animations
- Multiplayer features (compete with friends)
- Social sharing of achievements
- Custom workout creation
- Boss battles with special mechanics
- Equipment and inventory system
- Achievement badges and trophies
- Mobile app version
- Workout plans and schedules
- Integration with fitness wearables

5. Non-Functional Requirements

Performance

- The system shall load quickly under normal network conditions.
- API responses shall return within 500ms for standard operations.
- The system shall support at least 100 concurrent users on free tier hosting.

Security

- The system shall encrypt user passwords using bcrypt hashing.
-

- The system shall protect authenticated routes using JWT verification.
- The system shall sanitize user inputs to prevent injection attacks.
- The system shall not expose sensitive data in API responses.

Usability

- The system shall be user-friendly with intuitive navigation.
- The system shall provide clear feedback for all user actions.
- The system shall be accessible on desktop and mobile browsers.
- The system shall use consistent design patterns throughout.

Reliability

- The system shall be available 24/7 when deployed to cloud hosting.
- The system shall auto-recover from crashes (Railway auto-restart).
- The system shall gracefully handle errors with user-friendly messages.
- The system shall persist data even after the server restarts.

Scalability

- The system architecture shall support horizontal scaling if needed.
- The database schema shall allow for future feature additions.

Maintainability

- The code shall be well-documented with clear comments.
- The system shall use modular components for easy updates.
- API endpoints shall follow RESTful conventions.
- Everything must be maintained and properly taken care of on GitHub

6. System Architecture

The system follows a three-tier architecture:

Presentation Layer (User Interface)

1. Technologies: HTML5, CSS3, JavaScript, Tailwind CSS
2. Components:
 - Landing page (fitquest.html)
 - Login page (login.html)
 -

- Registration page (signup.html)

Dashboard (dashboard.html)

- Battle arena (battle.html)
- Training grounds (workout.html)

3. Features: Responsive design, interactive elements, real-time updates

Business Logic Layer (Application Server)

1. Technologies: Node.js, Express.js
2. Components:
 - Authentication middleware (JWT verification)
 - User management endpoints
 - Workout logging and XP calculation
 - Battle system logic and damage calculation
 - Quest management
 - Shop and inventory system

3. Features: RESTful API, input validation, error handling

Data Layer (Database)

1. Technologies: MongoDB with Mongoose ODM
2. Data Models:
 - User Schema (email, password, heroName, stats, level, XP, gold)
 - Workout Schema (name, reps, weight, XP, date)
 -

- Battle Schema (enemyName, HP, damage, outcome, date)
- Water Intake Schema (date, cups)

Quest Schema (title, description, type, requirement, rewards)

-

- Hosting: MongoDB Atlas (cloud) or Railway MongoDB

3. Features: Data persistence, indexing for performance, schema validation

Deployment Architecture

1. Hosting Platform: Railway.app (subject to change)
2. Environment: Node.js runtime with automatic SSL
3. Database: Cloud MongoDB with IP whitelisting
4. Scaling: Auto-scaling based on traffic
5. Monitoring: Railway dashboard with logs and metrics

Data Flow

1. User interacts with Presentation Layer (HTML/JS)
2. Frontend sends HTTP requests to Business Logic Layer (Express API)
3. Server processes requests and interacts with Data Layer (MongoDB)
4. Server returns JSON responses to frontend
5. Frontend updates UI based on response data

Architecture Diagram: [Diagram](#)